LOGO Video Text Spotting: System Architecture (Backend + Frontend)

Status: design for implementation in Sprint 7 (demo layer). The research pipeline (CLI, `src/logo`) stays the source of truth; the web layer only **wraps** it and never changes its file formats.

Stack decision (fixed, do not re-debate in each chat):

Layer	Choice	Reason
Research pipeline	Python 3.11, PyTorch 2.1.2 + CUDA 12.1 (`logo-torch`); detector may run in `logo-paddle`	Already set up; matches the CONTRACT
Backend API	FastAPI + Uvicorn, Pydantic v2, stdlib `sqlite3`	Small, typed, async upload and SSE
Job execution	One in-process worker thread + subprocess for the Paddle detector	One 6 GB GPU: one GPU job at a time
Storage	Filesystem (`runs/web-<job_id>/...`) + SQLite for job metadata	Reuses the stage file layout of the CONTRACT
Frontend	React 18 + TypeScript + Vite, TanStack Query, Recharts, plain CSS modules	Light, no heavy UI framework required
Video I/O	ffmpeg (H.264 proxy) + OpenCV	Browser-playable video with frame-accurate indices
Python package management	**uv** (`uv venv`, `uv pip`, optional `uv lock` / `uv sync`)	Fast, reproducible installs; same tool for backend, pipeline and Paddle env

---

1. System context

```
Browser (React SPA)
```

```
  |  HTTPS/HTTP  (REST + SSE + video range requests)
  v
FastAPI service  ---------------------------+
  |  API layer (routers, schemas, validation)|
  |  Job service (queue, state machine)      |
  |  Pipeline adapter (facade over logo.*)   |
  v                                          v
Worker thread (1 GPU job at a time)      SQLite (jobs.db)
  |
  |-- ffmpeg proxy + frame extraction        Filesystem
  |-- [subprocess] detector (paddle env, Plan A) or in-process OBB (Plan B)
  |-- LSC (ABINet)  -> fuse -> track -> vote -> export result.json
  v
runs/web-<job_id>/<stage>/web/input.<ext>   (same layout as research runs)
```

Design principles:

1. **Frozen contract.** The web layer reads and writes exactly the formats in the research CONTRACT (`det` , `lsc` , `fused` json, `tracks.jsonl` , `metrics.json`). It adds only `result.json` (UI-friendly view) and `job.json` .
2. **Stage-decoupled.** Every stage reads the previous stage's files; a failed job can resume from the last finished stage.
3. **Mock mode.** `LOGO_FAKE_PIPELINE=1` returns synthetic tracks so the frontend and API tests run with no GPU.
4. **Single GPU lock.** Jobs queue; exactly one runs. Peak VRAM stays inside the stage budgets measured in P23/P28.
5. **Untrusted input.** Uploads are validated by decoding, not by file extension; client file names are never used on disk.

---

2. Repository layout (additions only)

```
LOGO-VTS/
  src/logo/pipeline/api.py            # facade: run_stage(...)  (new, thin)
  backend/
    pyproject.toml                    # fastapi, uvicorn[standard], python-multip
    app/
      main.py                         # app factory, CORS, static mount, startup/
      settings.py                     # env-driven settings
      schemas.py                      # Pydantic models (Section 5)
      db.py                           # sqlite3 helpers, migrations
      deps.py                         # shared dependencies
```

```
    routers/
      health.py                    # GET /api/health
      configs.py                   # GET /api/configs
      jobs.py                      # POST/GET/DELETE jobs, events (SSE), files
      results.py                   # GET result, tracks.jsonl, export.csv
      experiments.py               # GET /api/experiments (research dashboard)
    services/
      job_service.py               # create, queue, state machine, cancel, cle
      worker.py                    # worker thread loop, GPU lock
      pipeline_adapter.py          # real adapter (calls logo.pipeline.api) +
      video_io.py                  # probe, validate, proxy transcode, frame e
      result_builder.py            # tracks.jsonl -> result.json
      events.py                    # in-memory pub/sub for SSE
  tests/
    test_upload_validation.py  test_state_machine.py  test_results.py  test_api
frontend/
  package.json  vite.config.ts  tsconfig.json  index.html
  src/
    main.tsx  App.tsx  routes.tsx
    api/{client.ts,types.ts,hooks.ts}
    pages/{DemoPage.tsx,JobPage.tsx,ExperimentsPage.tsx,AboutPage.tsx}
    components/
      UploadPanel.tsx  PresetPicker.tsx  JobStepper.tsx  VideoOverlayPlayer.tsx
      TrackList.tsx  FilterBar.tsx  TextSummaryTable.tsx  ExperimentTable.tsx
      ThresholdChart.tsx  ErrorBanner.tsx
    lib/{colors.ts,frames.ts,format.ts}
    styles/*.module.css
  tests/{frames.test.ts, overlay.test.ts}
configs/web_fast.yaml  configs/web_best.yaml        # presets shown in the UI
```

---

3. Backend architecture

3.1 Settings (environment variables)

Variable	Default	Meaning
`LOGO_ROOT`	repo root	Base path for `runs/`, `configs/`, weights
`LOGO_WEIGHTS_DIR`	`weights/`	detector, LSC, LST checkpoints (not in git)
		Interpreter of `logo-paddle` env

LOGO_PADDLE_PYTHON	empty	(Plan A); empty means Plan B in-process
LOGO_FAKE_PIPELINE	0	1 = synthetic results, no GPU
LOGO_MAX_UPLOAD_MB	200	Upload size limit
LOGO_MAX_DURATION_S	60	Longest accepted video
LOGO_MAX_LONG_SIDE	1280	Proxy resolution cap (long side, px)
LOGO_JOB_TTL_H	24	Delete finished job folders after this time
LOGO_CORS_ORIGINS	http://localhost:5173	Comma-separated allowed origins (dev)
LOGO_API_KEY	empty	If set, all /api calls need header X-API-Key

3.2 Job state machine

```
queued -> preparing -> detecting -> recognizing -> fusing -> tracking -> voting -
   |          |            |             |           |          |          |
   +----------+------------+-------------+-----------+----------+----------+-----
   any non-terminal state --(user)--> cancelling -> cancelled
```

- preparing : probe, validate, transcode to **constant-frame-rate H.264 proxy** (proxy.mp4), extract frames.
- detecting : detector -> det/web/input.json .
- recognizing : RoI-Rotate + LSC -> lsc/web/input.json + input_feats.npy .
- fusing : hard or soft fusion by preset -> fused/web/input.json .
- tracking : simple or LST tracker -> track/web/input.jsonl .
- voting : track-level voting/filtering -> vote/web/input.jsonl .
- exporting : build result.json , export.csv , preview thumbnail.
- Cancellation is cooperative: checked between stages and between batches via the cancel() callback.
- Resume: on startup, jobs left in a running state are marked failed with code SERVER_RESTART (simple and safe); user can resubmit.

3.3 Pipeline facade (only addition inside `src/logo`)

`src/logo/pipeline/api.py`:

```
from typing import Callable, Optional

ProgressCb = Callable[[int, int], None]     # (done, total)
CancelCb = Callable[[], bool]

STAGES = ("detect", "lsc", "fuse", "track", "vote")

def run_stage(stage: str, cfg: dict, ds: str, videos: list[str],
              progress: Optional[ProgressCb] = None,
              cancel: Optional[CancelCb] = None) -> None:
    """Runs one stage for the given videos, reading/writing
    runs/<cfg['exp_id']>/<stage>/<ds>/<video>.<ext> exactly as scripts/run_pipeli
    Raises PipelineCancelled if cancel() returns True between batches.
    The detect stage uses LOGO_PADDLE_PYTHON via subprocess when set, else the in
    """
```

Rules: the facade only calls existing modules (`logo.lsc.run_lsc`, `logo.scoring.run_fuse`, `logo.tracking.run_track`, `logo.voting.track_vote`, detector scripts). It adds progress and cancel hooks; it does not change formats. `scripts/run_pipeline.py` should call the same facade so CLI and web share one path.

3.4 Video preparation (`video_io.py`)

1. Save upload to `runs/web-<id>/upload.bin` using a streamed write with a size cap.
2. `ffprobe` (or OpenCV fallback) must decode the first frames successfully; reject if duration > `LOGO_MAX_DURATION_S`, no video stream, or zero frames.
3. Transcode to `proxy.mp4`: H.264, yuv420p, **constant** frame rate (source fps rounded, capped at 30), long side at most `LOGO_MAX_LONG_SIDE`, faststart flag for browser playback.
4. Extract frames from `proxy.mp4` to `runs/web-<id>/frames/web/input/%06d.jpg` (1-based). **Processing and playback use the same proxy**, so frame index `n` in results equals frame `n` in the browser.
5. Record `video` metadata (width, height, fps, num_frames) in `job.json`.
   ffmpeg is required for the web demo (browser-compatible output). If missing, `/api/health` reports it and uploads return `503 FFMPEG_MISSING`.

3.5 Result builder (`result_builder.py`)

Input: `vote/web/input.jsonl` (CONTRACT tracks format). Output: `result.json` (Section 5.3), grouping boxes by `track_id`, with per-track `text` (voted), `mean_score`, `first_frame`, `last_frame`, `num_frames`. Also writes `export.csv` (`track_id,first_frame,last_frame,num_frames,text,mean_score`).

3.6 Concurrency, limits, cleanup

- One worker thread consumes a FIFO queue; a `threading.Lock` guards the GPU section.
- Queue length limit (default 5). Beyond it: `429 QUEUE_FULL`.
- SSE events published through an in-memory pub/sub keyed by job id; clients that connect late receive the latest state first.
- Cleanup task (every 30 min): remove finished/failed job folders older than `LOGO_JOB_TTL_H`; keep `job.json` summary rows in SQLite.
- Disk guard: refuse new jobs when free disk is below 2 GB (`507 DISK_LOW`).

3.7 Security (local demo level, still do these)

- Ignore client filenames; generate `job_id = uuid4().hex`.
- Content validation by decoding; limit size and duration; strip metadata not needed.
- Paths built only from `job_id` after regex check `^[0-9a-f]{32}$`.
- CORS restricted to configured origins; optional `X-API-Key`.
- No arbitrary config upload: only presets from `configs/web_*.yaml` can be selected.
- Never serve files outside `runs/web-<id>/` (use explicit allow-list of file names).
- Uploaded videos may contain personal data: README states they are stored locally and auto-deleted after TTL.

3.8 Error model

All errors: `{"error": {"code": "STRING", "message": "human text", "stage": "optional"}}`.

HTTP	code	When
400	`INVALID_VIDEO`	Not decodable, no video stream
413	`FILE_TOO_LARGE`	Over `LOGO_MAX_UPLOAD_MB`
422	`DURATION_TOO_LONG` / `BAD_PRESET`	Validation
404	`JOB_NOT_FOUND`	Unknown id

409	`JOB_NOT_READY`	Result requested before `done`
429	`QUEUE_FULL`	Queue limit
503	`FFMPEG_MISSING` / `GPU_UNAVAILABLE`	Environment problem
500	`STAGE_FAILED`	A stage raised; `stage` field set; traceback only in server log

3.9 Database

```
CREATE TABLE IF NOT EXISTS jobs (
  id TEXT PRIMARY KEY,
  status TEXT NOT NULL,            -- queued|preparing|detecting|recognizing|fusi
  preset TEXT NOT NULL,            -- web_fast | web_best
  progress_done INTEGER DEFAULT 0,
  progress_total INTEGER DEFAULT 0,
  created_at TEXT NOT NULL,        -- ISO 8601 UTC
  started_at TEXT, finished_at TEXT,
  video_meta TEXT,                 -- JSON {width,height,fps,num_frames,duration_
  stage_timings TEXT,              -- JSON {stage: seconds}
  error TEXT                       -- JSON error object or NULL
);
CREATE INDEX IF NOT EXISTS idx_jobs_created ON jobs(created_at);
```

3.10 Presets (`configs/web_*.yaml`)

Both follow the research config keys. Differences:

Preset	Intent	Settings
`web_fast`	Quick demo	hard fusion, `tracker.type: simple` , voting off
`web_best`	Our final system	best fusion mode and `w` from validation, `tracker.type: lst` (or `embed` if P21F), voting on with tuned `tau` , `min_len`

Hyperparameters are copied from the **frozen** values chosen on validation (P25/P26/P28); the web layer never tunes anything.

---

4. REST API

Base path `/api`. JSON unless stated.

Method	Path	Purpose
GET	`/api/health`	GPU name, free VRAM, ffmpeg present, queue length, mode (`real` / `fake`)
GET	`/api/configs`	Presets: `[{name,title,description}]`
POST	`/api/jobs`	Multipart: `video` (file), `preset` (string). Returns `201 JobOut`
GET	`/api/jobs`	Recent jobs (limit 20)
GET	`/api/jobs/{id}`	`JobOut` (status, stage, progress, timings, error)
GET	`/api/jobs/{id}/events`	**SSE** stream of `JobEvent`
DELETE	`/api/jobs/{id}`	Cancel if running, else delete files; `204`
GET	`/api/jobs/{id}/video`	Proxy video, supports HTTP Range
GET	`/api/jobs/{id}/result`	`ResultOut` (409 until done)
GET	`/api/jobs/{id}/tracks.jsonl`	CONTRACT-format download
GET	`/api/jobs/{id}/export.csv`	Track table download
GET	`/api/experiments`	List research runs with headline metrics
GET	`/api/experiments/{exp_id}`	Full `metrics.json` of a run
GET	`/api/experiments/{exp_id}/sweep`	Threshold-sweep CSV parsed to JSON if present

SSE event payload (one JSON per `data:` line):

```
{"type":"stage","job_id":"...","status":"recognizing","done":120,"total":300,"ela
{"type":"done","job_id":"..."}
{"type":"error","job_id":"...","error":{"code":"STAGE_FAILED","message":"...","st
```

Send a heartbeat comment line every 15 s; the client reconnects with the browser's built-in `EventSource` retry.

---

5. Data contracts

5.1 Pydantic (backend/app/schemas.py)

```
from pydantic import BaseModel, Field
from typing import Literal, Optional

Status = Literal["queued","preparing","detecting","recognizing","fusing","trackin
                 "voting","exporting","done","failed","cancelled"]

class ErrorBody(BaseModel):
    code: str
    message: str
    stage: Optional[str] = None

class VideoMeta(BaseModel):
    width: int
    height: int
    fps: float
    num_frames: int
    duration_s: float

class JobOut(BaseModel):
    id: str
    status: Status
    preset: str
    progress_done: int = 0
    progress_total: int = 0
    created_at: str
    started_at: Optional[str] = None
    finished_at: Optional[str] = None
    video: Optional[VideoMeta] = None
    stage_timings: dict[str, float] = Field(default_factory=dict)
    error: Optional[ErrorBody] = None

class TrackOut(BaseModel):
    track_id: int
    text: str                                    # voted transcription (lowercase
    mean_score: float
```

```
    first_frame: int                            # 1-based, inclusive
    last_frame: int
    num_frames: int
    boxes: dict[str, list[float]]               # frame (string) -> [x1,y1,x2,y2,

class ResultOut(BaseModel):
    job_id: str
    preset: str
    video: VideoMeta
    tracks: list[TrackOut]
```

5.2 TypeScript (frontend/src/api/types.ts)

```
export type Status =
  | "queued" | "preparing" | "detecting" | "recognizing" | "fusing"
  | "tracking" | "voting" | "exporting" | "done" | "failed" | "cancelled";

export interface ErrorBody { code: string; message: string; stage?: string }
export interface VideoMeta { width: number; height: number; fps: number; num_fram

export interface JobOut {
  id: string; status: Status; preset: string;
  progress_done: number; progress_total: number;
  created_at: string; started_at?: string; finished_at?: string;
  video?: VideoMeta; stage_timings: Record<string, number>; error?: ErrorBody;
}

export interface TrackOut {
  track_id: number; text: string; mean_score: number;
  first_frame: number; last_frame: number; num_frames: number;
  boxes: Record<string, number[]>;           // frame -> 8 numbers (quad)
}
export interface ResultOut { job_id: string; preset: string; video: VideoMeta; tr

export type JobEvent =
  | { type: "stage"; job_id: string; status: Status; done: number; total: number;
  | { type: "done"; job_id: string }
  | { type: "error"; job_id: string; error: ErrorBody };
```

5.3 Example `result.json`

```
{
  "job_id": "3f9c...e1",
  "preset": "web_best",
```

```
  "video": {"width": 1280, "height": 720, "fps": 25.0, "num_frames": 250, "durati
  "tracks": [
    {"track_id": 3, "text": "coca", "mean_score": 0.83, "first_frame": 12, "last_
     "num_frames": 85,
     "boxes": {"12": [410,220,498,222,497,252,409,250], "13": [411,220,499,222,49
  ]
}
```


Frame indices are 1-based and refer to frames of `proxy.mp4`, identical to the research CONTRACT.

---

6. Frontend architecture

6.1 Pages and routing

Route	Page	Purpose
`/`	`DemoPage`	Upload, choose preset, recent jobs
`/jobs/:id`	`JobPage`	Progress stepper while running; player + tracks when done
`/experiments`	`ExperimentsPage`	Ablation table, threshold curves, efficiency (reads `/api/experiments`)
`/about`	`AboutPage`	Method summary (LSC, VPMM, LST, our C2/C3), limitations, links

6.2 Component responsibilities

Component	Responsibility
`UploadPanel`	Drag and drop, client checks (size/type) as hints only, progress bar for upload, calls `POST /api/jobs`
`PresetPicker`	Loads `/api/configs`; radio cards (Fast / Best) with a one-line description
`JobStepper`	Shows the pipeline stages with the current one highlighted, `done/total` and elapsed time; driven by SSE

`VideoOverlayPlayer`	`<video>` with a `<canvas>` overlay of quads, ids and text (Section 6.4)
`FilterBar`	Min score slider, min track length, toggles: ids, text, boxes
`TrackList`	Scrollable list sorted by length; click seeks video to `first_frame`; hover highlights the box
`TextSummaryTable`	All tracks (text, frames, score); CSV download button
`ExperimentTable`	Ablation rows from `/api/experiments`; marks `reproduced` runs; shows paper-reported numbers only if present in a config file
`ThresholdChart`	Recharts lines: MOTA and IDF1 vs threshold, hard vs soft
`ErrorBanner`	Maps API error codes to friendly messages

6.3 State and data fetching

- **Server state:** TanStack Query (`useJob(id)`, `useResult(id)`, `useConfigs()`, `useExperiments()`).
- **Live progress:** `useJobEvents(id)` wraps `EventSource`; on events it updates the `["job", id]` query cache; on `done` it invalidates `["result", id]`; on `error` it shows `ErrorBanner`. Fallback: poll `GET /api/jobs/{id}` every 2 s if SSE fails.
- **UI state (local, no global store needed):** current time/frame, selected track, filters.
- Upload uses `XMLHttpRequest` (or `fetch` with a stream wrapper) to report upload progress.

6.4 Overlay rendering (the one tricky part)

```
// lib/frames.ts
export const frameFromTime = (mediaTime: number, fps: number) =>
  Math.floor(mediaTime * fps + 0.5) + 1;               // 1-based, matches the CO

// build once when the result loads
export function indexByFrame(tracks: TrackOut[]) {
  const m = new Map<number, {id: number; text: string; score: number; quad: numbe
  for (const t of tracks)
    for (const [f, quad] of Object.entries(t.boxes)) {
      const k = Number(f);
      if (!m.has(k)) m.set(k, []);
      m.get(k)!.push({ id: t.track_id, text: t.text, score: t.mean_score, quad })
    }
```

```
  return m;
}
```

Rendering rules:

1. Draw on every presented video frame using `video.requestVideoFrameCallback` (use `metadata.mediaTime`); fall back to `requestAnimationFrame` where unsupported.
2. Canvas size equals the displayed video size (use `ResizeObserver`); scale quads by `displayWidth / video.width` and `displayHeight / video.height` from `VideoMeta`, not from `videoWidth` of the element, to avoid mismatch.
3. Quad drawn as a closed polygon; label (`#id text`) above the first vertex with a contrasting background; colour from `colors.ts` (`hsl(id * 47 % 360, 85%, 50%)`).
4. Paused/seek: redraw on `seeked` and `timeupdate`.
5. Filters apply before drawing: `score >= minScore`, `num_frames >= minLen`.
6. Because processing used the same constant-frame-rate proxy, no timestamp correction is needed.

6.5 API client (`api/client.ts`)

```
const BASE = import.meta.env.VITE_API_BASE ?? "/api";
async function j<T>(r: Response): Promise<T> {
  if (!r.ok) { const b = await r.json().catch(() => ({})); throw new ApiError(r.s
  return r.json();
}
export const api = {
  health: () => fetch(`${BASE}/health`).then(j),
  configs: () => fetch(`${BASE}/configs`).then(j),
  createJob: (file: File, preset: string, onProgress?: (p: number) => void) => /*
  job: (id: string) => fetch(`${BASE}/jobs/${id}`).then(j<JobOut>),
  result: (id: string) => fetch(`${BASE}/jobs/${id}/result`).then(j<ResultOut>),
  cancel: (id: string) => fetch(`${BASE}/jobs/${id}`, { method: "DELETE" }),
  videoUrl: (id: string) => `${BASE}/jobs/${id}/video`,
  csvUrl: (id: string) => `${BASE}/jobs/${id}/export.csv`,
  experiments: () => fetch(`${BASE}/experiments`).then(j),
};
```

Dev proxy (`vite.config.ts`): `server.proxy = { "/api": "http://localhost:8000" }`. If `LOGO_API_KEY` is set, the client adds header `X-API-Key` from `VITE_API_KEY`.

6.6 UX requirements

- Always show what is happening: stage, `done/total`, elapsed; allow Cancel.

- Empty and error states for: no text found, job failed, queue full, backend offline.
- Keyboard: space = play/pause, left/right = previous/next frame (pause first), `t` toggles text labels.
- Responsive down to tablet width; the video never overflows its container.
- Accessible: buttons labelled, sufficient contrast, no information by colour alone (ids printed).
- Show a short disclaimer on `AboutPage` : results come from a research prototype; accuracy figures belong to the validation protocol of the report.

---

7. Experiments dashboard (research view)

Backend reads `runs/*/metrics.json` (CONTRACT format) and optional `runs/*/sweep.csv` :

`GET /api/experiments` returns:

```
[{"exp_id":"E5-lst-v1","dataset":"icdar2015v","split":"val",
  "track":{"MOTA":0.0,"IDF1":0.0},"spot":{"MOTA":0.0,"IDF1":0.0},"created":"2026-
```

(values come from files; the example zeros are placeholders). Frontend shows: ablation table (selected runs in the order of `configs/ablation.yaml`), hard vs soft curves, and efficiency (`FINAL-efficiency.json`). Numbers are displayed exactly as stored; nothing is computed or typed by hand in the UI. The page is read-only and works without a GPU.

---

8. Build, run, deploy

Development (two terminals):

```
# backend (inside the logo-torch env, managed with uv)
uv pip install -e . && uv pip install -e backend
LOGO_FAKE_PIPELINE=1 uvicorn app.main:app --reload --app-dir backend --port 8000

# frontend
cd frontend && npm install && npm run dev        # http://localhost:5173 (proxy t
```

Single-process "demo mode" for the review:

```
cd frontend && npm run build                      # creates frontend/dist
LOGO_PADDLE_PYTHON=/path/to/logo-paddle/python \
```

```
LOGO_WEIGHTS_DIR=weights \
uvicorn app.main:app --app-dir backend --host 127.0.0.1 --port 8000
# FastAPI mounts frontend/dist at "/" and the API at "/api"
```

Health check before the demo: open `/api/health` and confirm `mode: real`, GPU name, ffmpeg present, queue 0. Pre-run three videos and keep their job pages as backups.

Package management with uv

- Create environments with uv: `uv venv --python 3.11 .venv` (pipeline + backend) and `uv venv --python 3.10 .venv-paddle` (detector, Plan A). uv can download the requested Python if it is missing.
- Install with `uv pip install ...` (never plain `pip`). Declare dependencies in `pyproject.toml` (root package `logo`, and `backend/pyproject.toml`); record exact versions with `uv pip freeze > requirements-torch.txt` and `requirements-paddle.txt` after each milestone.
- PyTorch stays on the CUDA 12.1 build already verified (`uv pip install torch==2.1.2 --index-url https://download.pytorch.org/whl/cu121` if you ever recreate the env). PaddlePaddle-GPU uses the index URL from the official install selector; pass it to uv with `--index-url` (or `--extra-index-url`). Do not guess the URL.
- Optional: once stable, add a lockfile (`uv lock`) for the backend and use `uv sync` on a new machine. Keep the two-environment split; do not merge Paddle into the torch env.
- Run tools through the env, for example `uv run --no-sync pytest -q` or by activating `.venv`.
- The frontend is separate (`npm` / `pnpm`); uv is for Python only.

---

9. Testing strategy

Level	What	Tool
Backend unit	upload validation, state machine transitions, result builder, error model	pytest
Backend API	full job lifecycle in fake mode (upload -> SSE -> result -> csv -> delete), 429/413/409 cases	pytest + httpx `TestClient`
Pipeline contract	facade writes the same files as the CLI; `tracks.jsonl`	pytest (small real clip, optional)

	parses into `ResultOut`	
Frontend unit	`frameFromTime`, `indexByFrame`, filter logic	Vitest
Frontend component	`VideoOverlayPlayer` draws expected number of polygons for a given frame (canvas mocked)	Vitest + Testing Library
End to end (optional)	upload sample video in fake mode and see overlay	Playwright

Acceptance: `pytest -q backend/tests` and `npm test` pass without a GPU; one real-mode run on a never-seen clip completes and the overlay matches the boxes in `tracks.jsonl`.

---

10. Implementation order and paste-ready prompts

Prerequisites: P13 to P18 done (modules exist) and ideally P22 and P26 (tracker and voting). Paste **Block A** from the AI Prompt Pack plus `PROJECT_STATE.md` first; then add this file's sections as noted.

P34: Pipeline facade and backend core. Provide: Sections 1 to 5 and 8 of this file; the CLI entry `scripts/run_pipeline.py`.

```
TASK P34. Implement src/logo/pipeline/api.py (Section 3.3) and refactor scripts/r
```

Accept when: `pytest -q backend/tests` passes with no GPU; `GET /api/health` works; fake job completes through SSE.

P35: Frontend app. Provide: Sections 4 to 6 of this file; the running fake backend.

```
TASK P35. Create frontend/ (React 18 + TypeScript + Vite) exactly as in Section 2
```

Accept when: `npm test` passes; in fake mode you can upload a video, watch the stepper progress, and see boxes with ids and text follow the video; filters and CSV download work.

P36: Experiments dashboard. Provide: Section 7; a few real `runs/*/metrics.json`.

```
TASK P36. Add routers/experiments.py (Section 4 and 7) and frontend ExperimentsPa
```

Accept when: dashboard shows your ablation rows and hard-vs-soft curves from stored

files.

P37: Real-mode integration and polish. Provide: your env paths and a short real test clip.

```
TASK P37. Switch the adapter to real mode: detect via LOGO_PADDLE_PYTHON subproce
```

Accept when: a never-seen clip runs end to end in real mode, overlay matches `tracks.jsonl`, and the pre-demo checklist passes.

---

11. Non-goals and limits (state these honestly in the report)

- No multi-user accounts, no cloud scaling, one GPU job at a time.
- Demo accuracy equals the validation protocol of the project report; the demo is not a benchmark.
- Long or very high-resolution videos are intentionally limited by the proxy step (defaults above).
- Throughput numbers are not promised here; use the measurements from P23/P28.

12. Decision log (add to docs/decisions.md)

Decision	Choice	Alternative kept in reserve
API framework	FastAPI	Flask (if FastAPI conflicts with env)
Job execution	Worker thread + subprocess for Paddle	Redis/RQ if ever multi-GPU
Progress channel	SSE with polling fallback	WebSocket
Video alignment	CFR H.264 proxy used for processing and playback	Timestamp mapping per frame
Frontend	React + Vite + TS, CSS modules	Streamlit/Gradio page if time runs out (see P30 `--gradio`)